



Непрерывная разработка и интеграция CI/CD

...

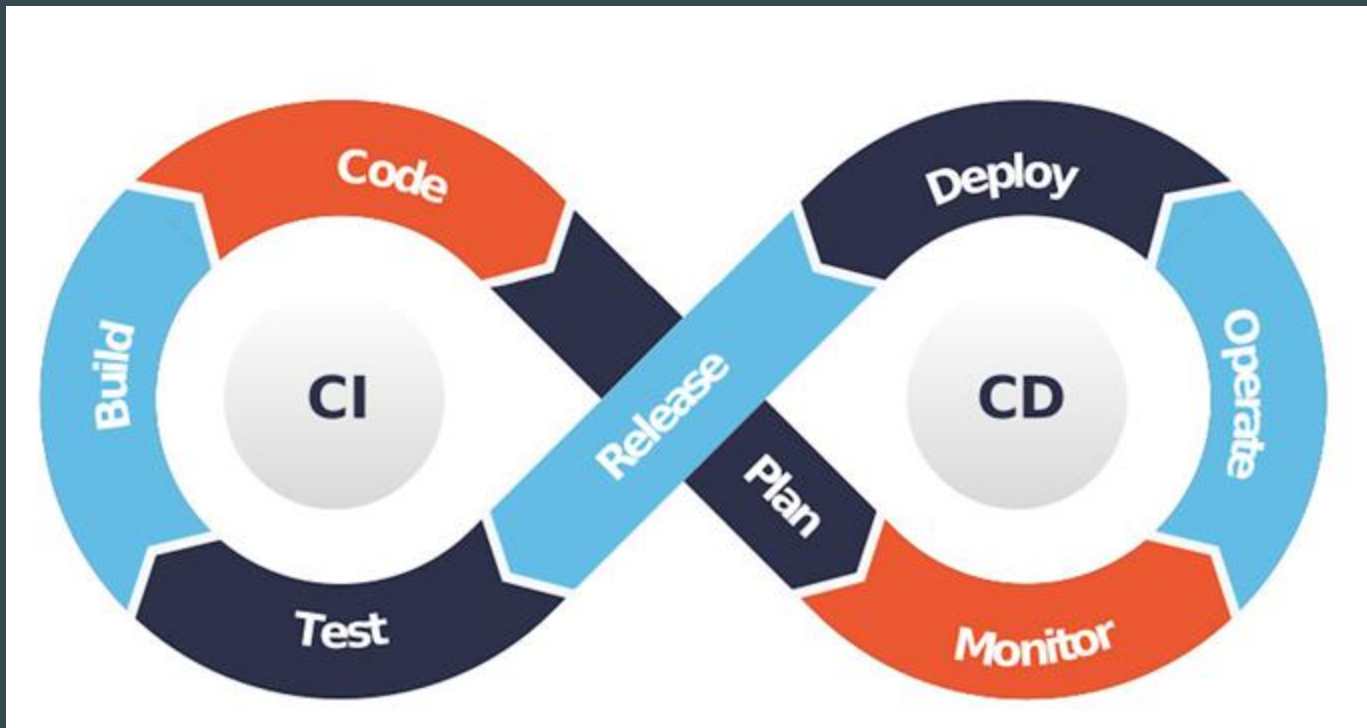
Лекция 2

Волков Михаил Юрьевич

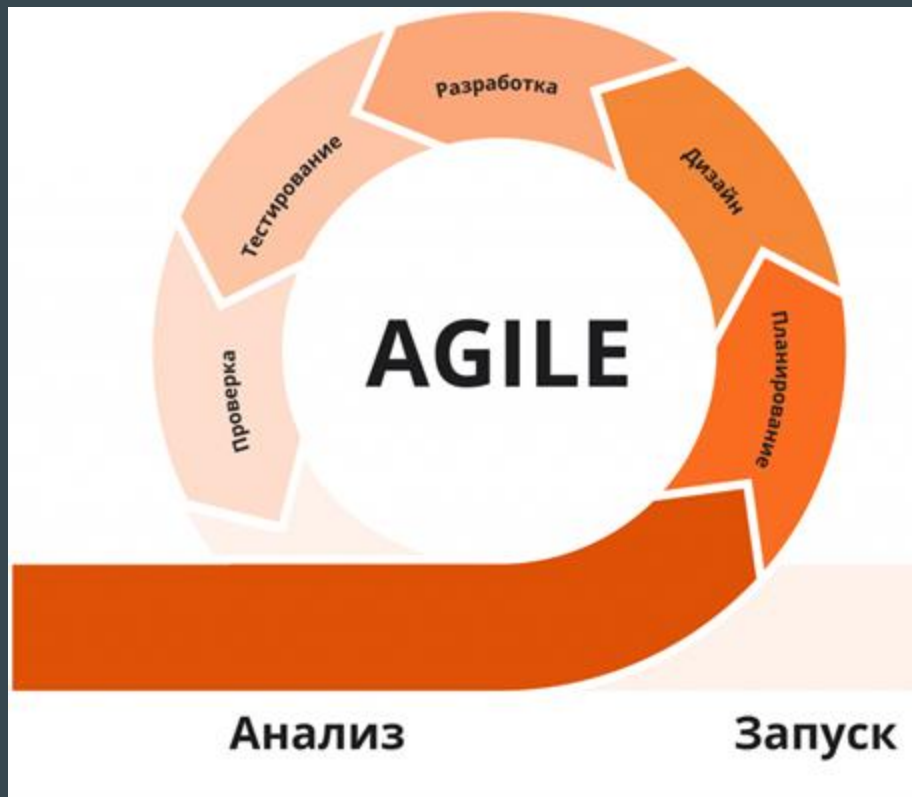
Тема 2

Общий принцип работы CI/CD

Роль CI/CD в DevOps



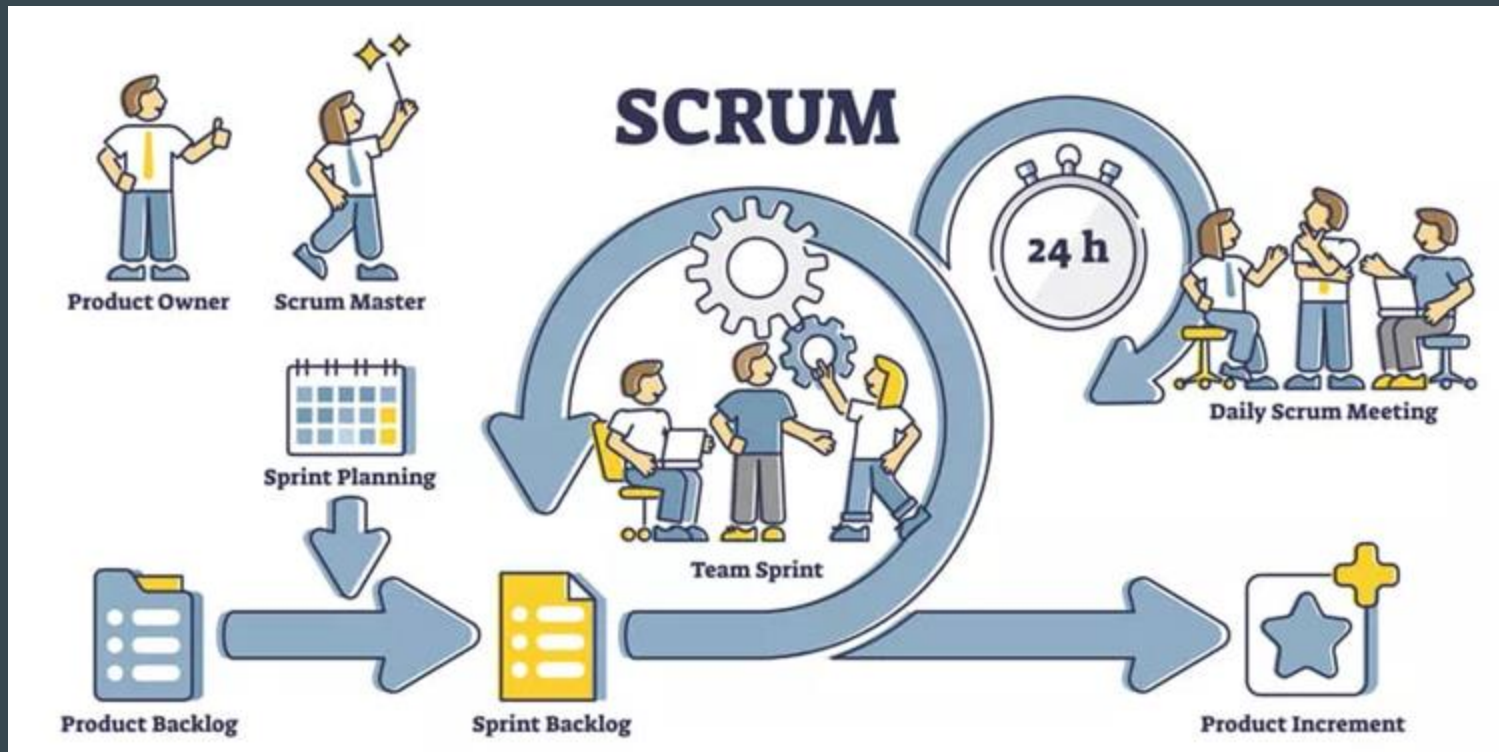
Agile – переворот в разработке ПО



Agile-манифест – декларация ценностей

Люди и взаимодействие	важнее	процессов и инструментов
Работающий продукт	важнее	исчерпывающей документации
Сотрудничество с заказчиком	важнее	согласования условий контракта
Готовность к изменениям	важнее	следования первоначальному плану

Agile – переворот в разработке ПО



Основные положения DevOps

Программисты охотно принимали принципы Agile и меняли подходы к разработке ПО, однако практически не работали совместно с командами, отвечающими за следующие этапы.

Поэтому зачастую команды Ops-специалистов, отвечающие за инфраструктуру и развертывание ПО в производственную среду, придерживались совершенно иных принципов.



Построение хорошего CI-процесса

1. Использование контроля версий
2. Делайте коммиты рано и часто
3. Выполнение сборки при каждом коммите
4. Автоматизирование тестов
5. Получение обратной связи
6. Создайте DevOps-культуру

Непрерывная доставка

Непрерывная доставка основывается на автоматизации сборки и тестирования, которую вводит непрерывная интеграция.

Данный подход имеет ряд преимуществ:

1. Избегая традиционной разобщенности, ваша команда будет лучше понимать бизнес-потребности и операционные нужды, возникающие при доставке продукта пользователям.
2. Позволяет привнести некоторые практики разработки в процесс, который принято считать ручным и довольно длительным.
3. Применение автоматизации не только ускоряет процесс доставки, но и снижает вероятность возникновения ошибок, делая его более стабильным и надежным.

Непрерывная доставка

Реализация непрерывной доставки:

1. Выполняйте сборку однократно.
2. Автоматизируйте развертывание.
3. Отделите параметры окружения.
4. Храните конфигурацию в VCS.
5. Чистите ваши окружения.
6. Поддерживайте пайплайн.

Непрерывное развертывание

Непрерывное развертывание завершает логическую цепочку из практик непрерывной интеграции и доставки: если сборка успешно проходит все предыдущие стадии пайплайна, она автоматически выпускается в продакшн.

Непрерывное развертывание подойдет не любому продукту и не любой организации. Однако имеет смысл рассмотреть шаги, которые оно включает, поскольку каждый из них способен нести ценность сам по себе:

1. Будьте уверены в своих тестах.
2. Следите за продакшном.
3. Выбирайте, что релизить.
4. Оптимизируйте пайплайн.
5. Контролируйте выпуск.

Конвейер развертывания — общий принцип CI/CD

Конвейер развертывания - Это реализация автоматизированных процессов сборки, установки, тестирования и поставки релиза. Конвейеры развертывания могут быть организованы по-разному, в зависимости от технологических особенностей поставки релизов, однако фундаментальные принципы непрерывного развертывания одинаковы в любых ситуациях.

Стадия фиксации

Компиляция
Модульные тесты
Анализ
Инсталляторы сборок

Автоматический приемочный тест

Автоматический тест производительности

Ручное тестирование

Демонстрация
Исследовательское
тестирование

Релиз

Пайплайны



Этапы билд-пайплайна

Процесс начинается с коммита в ветку `master` или любую другую ветку, выбранную вами для непрерывной интеграции. После коммита запускается либо сборка, либо первый набор юнит-тестов. Результаты передаются на панель мониторинга. Если при выполнении сборки или теста произошла ошибка, автоматически появляется уведомление об этом.

На следующем этапе выполняется ряд автоматических тестов, и после каждого раунда тестирования обрабатывается обратная связь.



Этапы билд-пайплайна

После завершения автоматических тестов обычно выполняется развертывание ПО в тестовых средах. Некоторые из них могут использоваться для дополнительного ручного тестирования, другие – для обучения, поддержки и предварительного знакомства пользователей с ПО.

Последний этап CI/CD-пайплайна включает в себя внесение изменений в режиме реального времени. Он может запускаться вручную (если реализована непрерывная доставка) или автоматически (при непрерывном развертывании).

Этапы билд-пайплайна

Рассмотрим некоторые аспекты каждого этапа:

1. Флаги и ветки – первый шаг к внедрению непрерывной интеграции — перенос всей кодовой базы в систему контроля версий, например в Git, Mercurial или Perforce. После этого всем членам команды нужно выработать привычку делать коммиты изменений чаще. Каждый коммит в ветку master запускает пайплайн, сборку и тестирование нового кода, позволяя быстро получить обратную связь.



mercurial

Этапы билд-пайплайна

2. Сборки и тесты - После запуска нового экземпляра пайплайна при коммите следующие этапы – сборка и тестирование. Автоматизированные юнит-тесты, как правило, запускаются перед сборкой вместе с проверками кода и статическим анализом.
3. Контейнеры и виртуальные машины - Чтобы гарантировать надежность результатов автоматических тестов, их необходимо выполнять на единообразной основе.
4. Предпроизводственные среды - Количество тестовых и предпроизводственных сред в пайплайне зависит от того, какое ПО вы разрабатываете, и от потребностей разных заинтересованных сторон в организации.
5. Развертывание - Когда изменения в коде успешно прошли все предыдущие этапы пайплайна, их можно развернуть в производственной среде. Последний этап может выполняться вручную или автоматически.

Оптимизация разработки

CI/CD повышает эффективность разработки ПО, помогая как можно раньше выявлять проблемы. За счет более быстрого взаимодействия и получения обратной связи удастся устранять ошибки на раннем этапе. Реализовать это можно благодаря автоматизации пайплайна.

При проектировании собственного пайплайна ПО лучше делать это поэтапно, начиная с непрерывной интеграции. Конкретные этапы и логика запуска каждого из них зависят от продукта и организации работы.

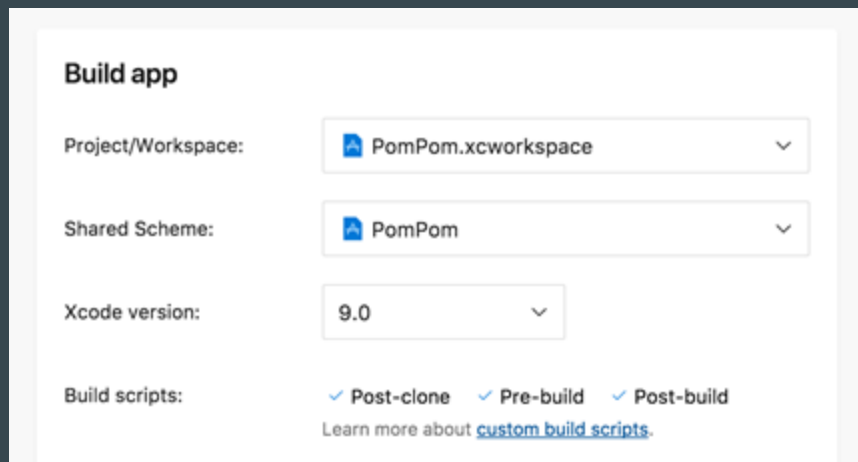
Основные принципы автоматизации сборки

Автоматическая сборка играет главную роль в непрерывной интеграции (CI) и является важным фактором в реализации CI/CD-пайплайна.

Файлы, созданные на этапе сборки (артефакты сборки), затем проходят следующие этапы CI/CD-пайплайна: тестирование и развертывание в тестовом окружении. После успешного прохождения всех этапов пайплайна сборка готова к развертыванию в производственную среду.

Автоматизация сборки в CI/CD-пайплайне

Настройка автоматической сборки и ее запуска каждый раз, когда кто-то из членов команды делает коммит изменений – одна из первых вещей, которые необходимо сделать (сразу после загрузки всех данных в систему контроля версий). Стоит прибегать к билд-скриптам или файлам определения IDE или написанных самостоятельно.



The image shows a 'Build app' configuration panel with the following settings:

- Project/Workspace:** PomPom.xcworkspace
- Shared Scheme:** PomPom
- Xcode version:** 9.0
- Build scripts:** ☒ Post-clone ☒ Pre-build ☒ Post-build

Below the build scripts, there is a link: [Learn more about custom build scripts.](#)

Запуск сборки

Непрерывная интеграция подразумевает, что каждый коммит в ветку master становится триггером сборки. В результате все изменения интегрируются и тестируются вскоре после того, как будут сделаны. Успешное завершение сборки служит триггером следующего этапа процесса.

Большинство инструментов для CI позволяют задать дополнительные триггеры сборки и настроить следующие этапы пайплайна.

Выполнение сборки

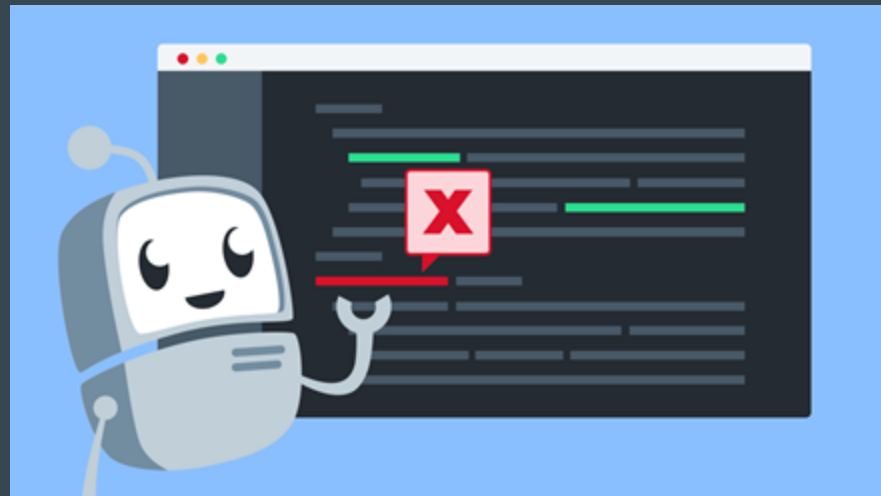
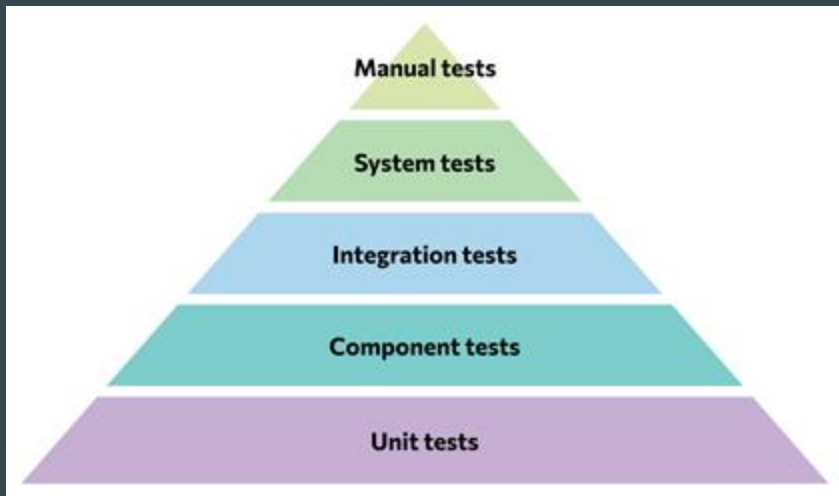
Рекомендуется выполнять все шаги сборки на выделенном билд-сервере, а не на машине разработчика. Если сборка выполняется в чистой среде, это поможет сразу найти отсутствующие зависимости и избежать в дальнейшем проблем.

На этапе сборки происходит вызов выбранного инструмента (например, Maven, Ant или Gradle), который и выполняет задачи, определенные в билд-скрипте или файле определения.



Выполнение тестов

Помимо подготовки кода к развертыванию, автоматическая сборка — идеальный момент для проведения ряда других проверок кода: например, юнит-тестирования, линтинга и статического анализа кода. Эти проверки стоит выполнять при каждой сборке, чтобы сразу же устранить обнаруженные проблемы. Это поможет повысить качество кода.



Публикация артефактов сборки

Результатом автоматической сборки становятся артефакты сборки. Они могут включать в себя установщики, WAR-файлы, библиотеки и контейнеры. Публикуя эти файлы в репозиторий артефактов, вы получаете централизованное хранилище, откуда можно развернуть сборки в разные среды.

Метрики и обратная связь

Инструмент непрерывной интеграции позволяет посмотреть результаты сборки, в том числе понять, успешно ли она завершилась, а также результаты юнит-тестирования и других проверок. Настроив уведомления о возможных сбоях, вы сможете быстро отреагировать на возникшие проблемы и восстановить рабочее состояние кода.



Автоматизированное тестирование для CI/CD

Задача непрерывной интеграции и развертывания (CI/CD) — позволить командам разработчиков часто выпускать пользователям работающее ПО, таким образом принося им пользу и узнавая, как продукт используется в реальном мире. Чтобы не отставать от конкурентов, многие компании начали использовать практику DevOps.



Использование автоматизированных тестов в CI/CD

При каскадной разработке ручное тестирование (или стадия QA) выполнялось уже после того, как код был написан и интегрирован, с целью проверить, соответствует ли поведение приложения спецификациям или нет.

Такой линейный подход замедляет процедуру релиза. Работает ли новый код, разработчик узнает с большой задержкой, когда поверх этого кода уже появится много другого. Процесс CI/CD, напротив, использует гибкий подход с короткими итеративными циклами, обеспечивающими быструю обратную связь и позволяющими часто выпускать небольшие обновления.

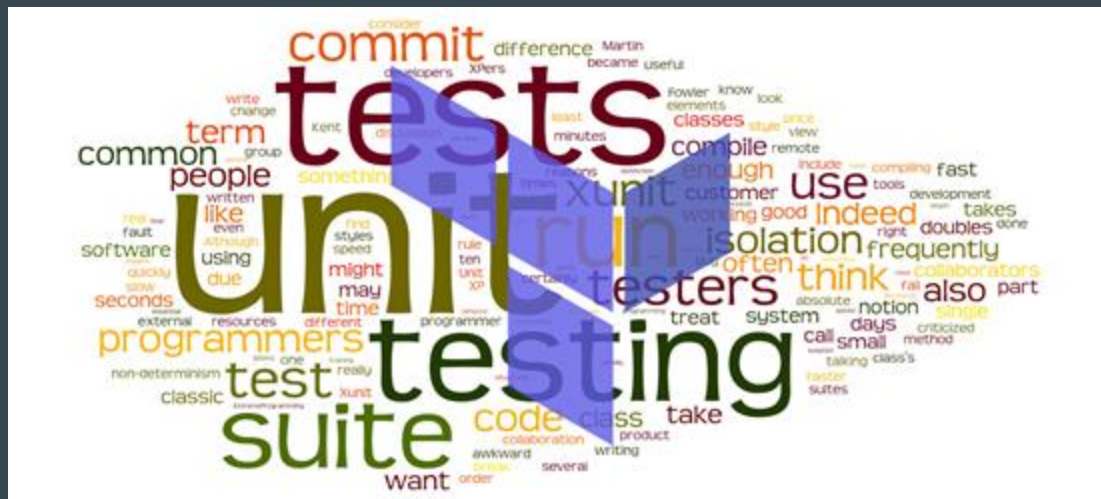
Создание пирамиды тестирования

Пирамида тестирования помогает составить представление о том, как приоритизировать тесты в CI/CD-пайплайне, определить нужное количество тестов и порядок их выполнения.

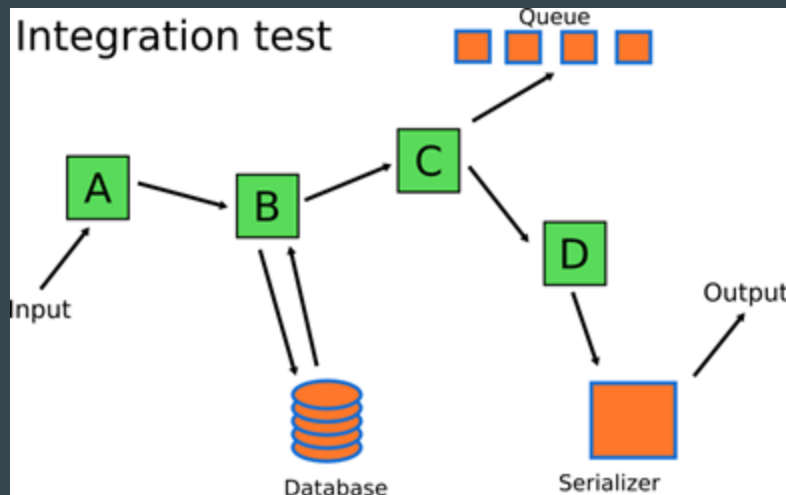
Название «пирамида тестирования» — не слишком точное, но исходная идея ясна: начать стоит с создания прочной базы автоматизированных юнит-тестов, которые быстро и легко выполнить, после чего можно перейти к более сложным для написания и долгим при выполнении тестам, а закончить — небольшим количеством наиболее сложных тестов.

Юнит-тесты

Юнит-тесты по праву занимают место в основании пирамиды. Эти тесты призваны проверить, что ваш код работает ожидаемым образом, адресуя как можно более мелкие составляющие поведения кода. Юнит-тесты, как правило, пишут разработчики — параллельно с написанием самого кода. Это естественным образом вытекает из методологии разработки через тестирование (Test Driven Development, TDD). Однако чтобы писать юнит-тесты, вовсе не обязательно следовать TDD.



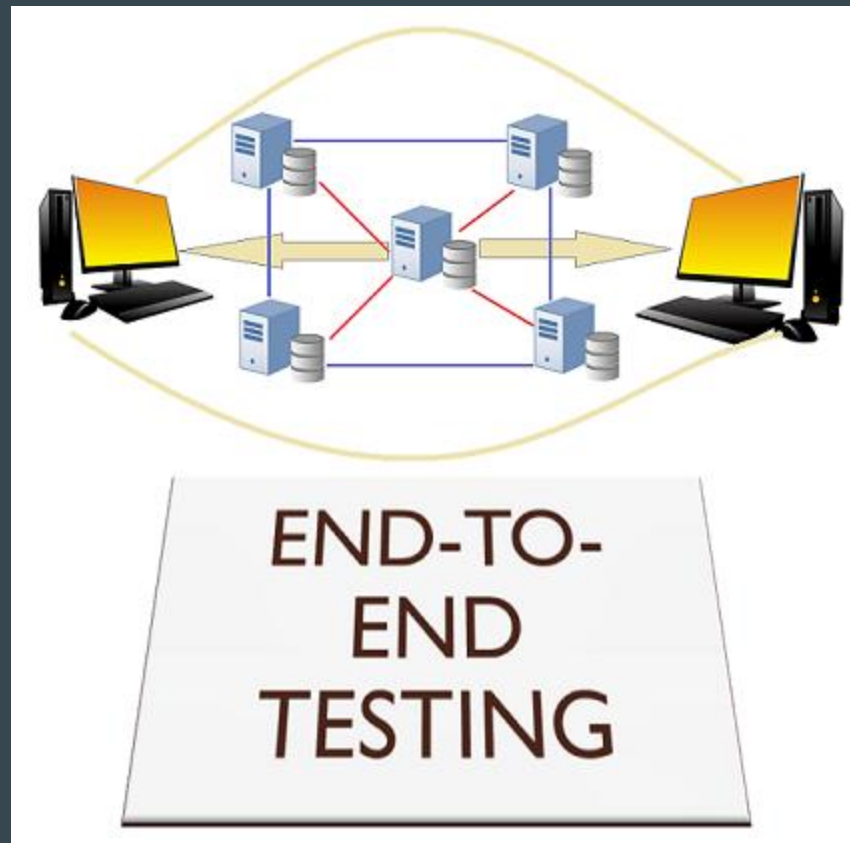
Интеграционные тесты



Интеграционные тесты позволяют убедиться, что компоненты ПО правильно взаимодействуют друг с другом (например, взаимодействие кода приложения с базой данных). Полезно разделить интеграционные тесты на широкие и узкие. В узких интеграционных тестах взаимодействие с другим модулем проверяется при помощи тестовых двойников, а не реального модуля, в то время как широкие интеграционные тесты взаимодействуют с реальной компонентой или сервисом.

Сквозные тесты

Также известные под названием full-stack, сквозные тесты охватывают приложение целиком. Эти тесты можно выполнять не только через пользовательский интерфейс, но и через API, которое тоже работает с несколькими компонентами системы (API также тестируют при помощи интеграционных тестов). Пирамида тестирования предписывает делать меньшее количество этих тестов, не только потому, что они дольше выполняются, но и потому, что они обычно более хрупкие.



Тестирование производительности

Под тестированием производительности понимается целый ряд стратегий тестирования для проверки того, как ваше ПО будет вести себя в реальной среде. Нагрузочное тестирование проверяет поведение системы при росте спроса. Стресс-тестирование намеренно превышает ожидаемый спрос. Тестирование на выносливость замеряет производительность при продолжительной высокой нагрузке.



Тестовые окружения

И тестам производительности, и сквозным тестам нужны тестовые окружения, как можно более точно повторяющие продакшн-окружение, и могут потребоваться тестовые данные. Чтобы автоматизированные тесты действительно вселяли уверенность в разрабатываемом ПО, важно, чтобы тесты каждый раз запускались одинаковым образом, в частности, чтобы тестовые окружения сохраняли одно и то же состояние от запуска к запуску (при этом они должны обновляться в соответствии с изменениями в продакшне).

Обработка обратной связи



Задача запуска автоматизированных тестов в рамках CI/CD в том, чтобы быстро получать обратную связь по изменениям в коде. Поэтому важно следить за поступающей информацией и реагировать на нее.

Руководство по инструментам CI/CD

Задача DevOps и CI/CD — помочь командам разработчиков быстрее создавать продукты без потери качества.

Инструменты непрерывной интеграции и непрерывного развертывания (CI/CD) играют ведущую роль в координации и автоматизации работы на различных этапах пайплайна: от запуска процесса после коммита до управления сборкой, запуска автоматических тестов, публикации артефактов и обработки полученной обратной связи.

Основная задача CI/CD-инструмента — координировать деятельность множества систем: системы контроля версий (например, Git, Mercurial или Perforce), инструментов сборки (например, Ant, Maven или Gradle), фреймворков автоматизированного тестирования (например, NUnit или JUnit), менеджеров зависимостей (например, NuGet), баг-трекеров (например, Jira, YouTrack или Github) и платформ контейнеризации (например, Docker).

Руководство по инструментам CI/CD

Поддержка отправки уведомлений по электронной почте и интеграции с IDE или коммуникационными платформами (например, Slack) позволяет получать информацию о том, что происходит в пайплайне, не заходя в панель мониторинга. При этом полезно иметь возможность гибкой настройки уведомлений: если их будет слишком много, они превратятся в фоновый шум.

Поддержка отправки уведомлений по электронной почте и интеграции с IDE или коммуникационными платформами (например, Slack) позволяет получать информацию о том, что происходит в пайплайне, не заходя в панель мониторинга. При этом полезно иметь возможность гибкой настройки уведомлений: если их будет слишком много, они превратятся в фоновый шум.

Спасибо за внимание !

